esprit Se former autrement HONORIS UNITED UNIVERSITIES	**EXAMEN** Semestre : 1 ■ 2 ☐ Session : Principale ■ Rattrapage ☐

Module: Architecture Orientée Services (SOA)
Enseignant(s) :UP_WEB
Classe(s) : 4TWIN, 4SE et 4SAE
Documents autorisés : OUI ☐ NON ■ Nombre de pages : 7 + 3 (**Feuille de réponse**)
Calculatrice autorisée : OUI ☐ NON ■ Internet autorisée : OUI ☐ NON ■
Date : 20-01-2022 Heure:15h00 Durée : 1H30 min

NB : Les étudiants sont demandés de répondre directement sur **la feuille de réponse**.

I. Partie 1 : QCU (6pts)

1) Dans une architecture Orientée Service, le consommateur et le fournisseur de service:

a. Doivent avoir la même architecture physique
b. Peuvent avoir des architectures différentes **à condition** qu'ils communiquent sur le même réseau
c. Ne sont pas obligés d'utiliser la même architecture physique
d. Ne sont pas obligés d'utiliser la même architecture physique tant qu'ils utilisent le même langage de programmation

2) Pour tester un SW étendu :

a. Il suffit d'avoir l'adresse (URL) du contrat WSDL
b. Il faut avoir l'adresse du contrat et obligatoirement le nom du service
c. Il faut connaitre les noms des opérations au préalable
d. Toutes les réponses sont correctes

3) Dans quel élément XML, du contrat, on trouve la liste des opérations exposées par le service :

a. <operations>
b. <portTpe>
c. <definition>
d. <service>

4) Soit le schema XML suivant :

```
<xs:element name="produit" type="produitType" />
<xs:[1]....  name ="produitType" >
    <xs:attribute name="ref" type="refType"  />
</xs:[1]....>
<xs:[2].... name ="refType" >
    <xs:restriction [3]....="xs:string">
      <xs:[4]....  value="[A-Z]{3} [a-z0-9]+"/>
    </xs:restriction>
</xs:[2]....>
```

On cherche à terminer le XSD ci-dessus, quelle proposition contient les bonnes réponses aux valeurs manquantes :

a. [1] :simpleType | [2] : complexType | [3] : type | [4] : pattern
b. [1] :complexContent | [2] : simpleContent | [3] : type | [4] : pattern
c. [1] :complexContent | [2] : simpleContent | [3] : base | [4] : text
d. [1] :complexType | [2] : simpleType | [3] : base | [4] : pattern

5) Dans un contrat WSDL, le nombre des éléments <message> dépend :

a. Du nombre des opérations exposées par le service,
b. Du besoin du client, et donc du nombre des opérations qu'il prévoit invoquer
c. Du protocole de communication utilisé
d. Les 3 réponses sont correctes

6) Soit la classe java suivante :

```
--------------
public class Article{
@XmlAttribute
public String ref;
@XmlTransient
public float prix;
}
```

Quel document XSD définit la grammaire adéquate ?

a.

```
<xs:complexType>
        <xs:sequence>
                <xs:attribute name = "ref"   type = "xs:string" />
        </xs:sequence>
</xs:complexType>
```

b.

```
<xs:complexType>
        <xs:sequence>
              <xs:element name="prix"    type="xs:float" />
        </xs:sequence>
       <xs:attribute name = "ref"    type = "xs:string" />
</xs:complexType>
```

c.

```
<xs:simpleType>
        <xs:attribute name = "ref" type = "xs: string" />
</xs:simpleType>
```

d.

```
<xs:complexType>
    <xs:attribute name = "ref" type = "xs:string" />
</xs:complexType>
```

II. Partie 2 : JAX-WS & GraphQL (14pts)

Etude de cas: Location de Voitures

Une agence de location de voiture vous a proposé de mettre en place une application Java pour la gestion de la liste de ses voitures de location. Cette application serait exposée en tant qu'un service Web afin de faciliter la communication avec d'autres applications hétérogènes.

L'application prend en charge deux entités ainsi qu'une énumération comme illustré dans la figure suivante:

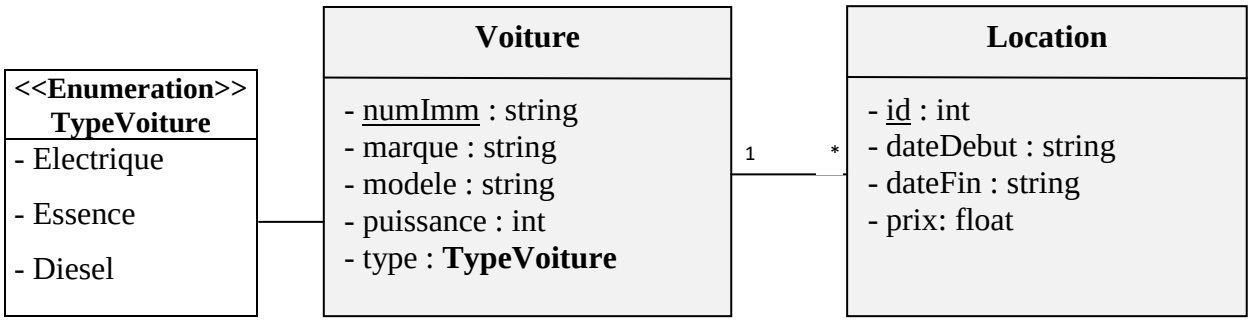


Figure1 : Diagramme de Classes

1. Exercice1: JAX-WS (7 pts)

On vise à développer en premier lieu un service web étendu pour gérer les voitures et les différentes opérations de location.

Les attributs ***numImm*** et ***id*** représentent respectivement les clés primaires des entités **Voiture** et **Location**. **TypeVoiture** est une énumération avec trois choix de valeurs possibles.

Ci-dessous les messages SOAP utilisées pour invoquer quelques opérations exposées par le SW proposé.

- **Ajout d'une nouvelle voiture :**

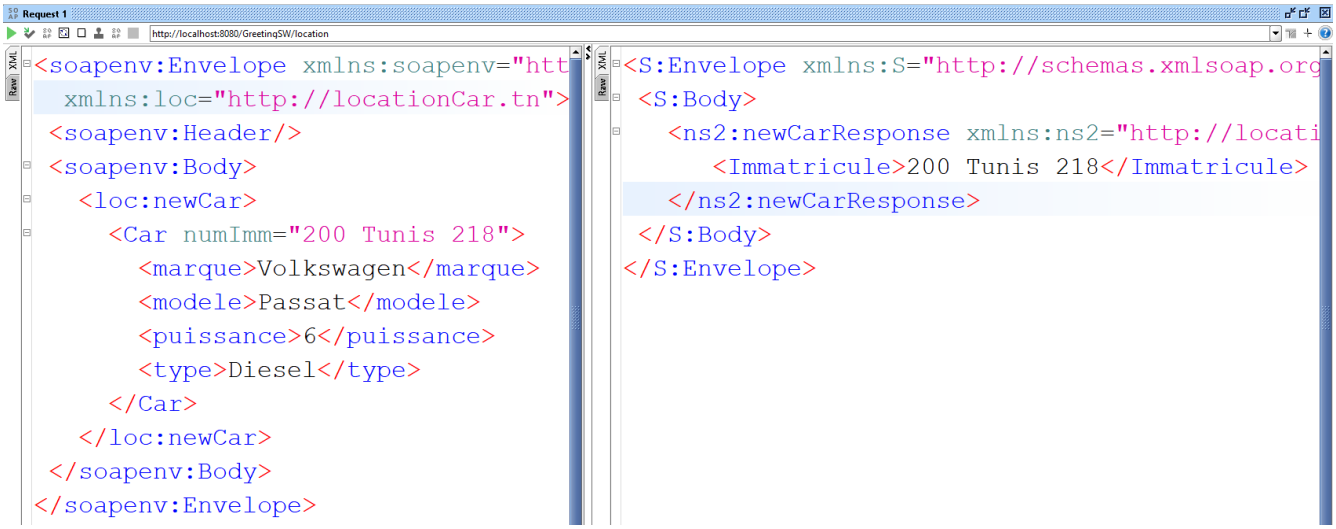


Figure 2: Messages SOAP Ajout Voiture

- **Détails locations :**

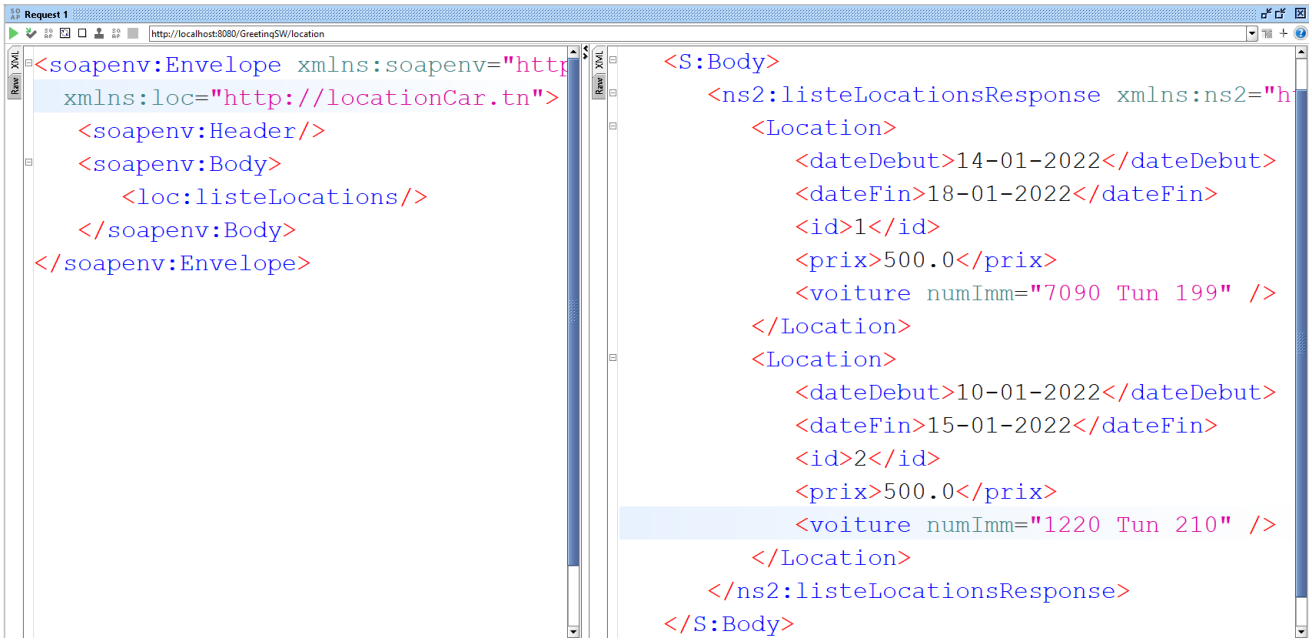


Figure 3: Messages SOAP Détails Locations

- **Voitures louées entre date début et date fin spécifiées :**

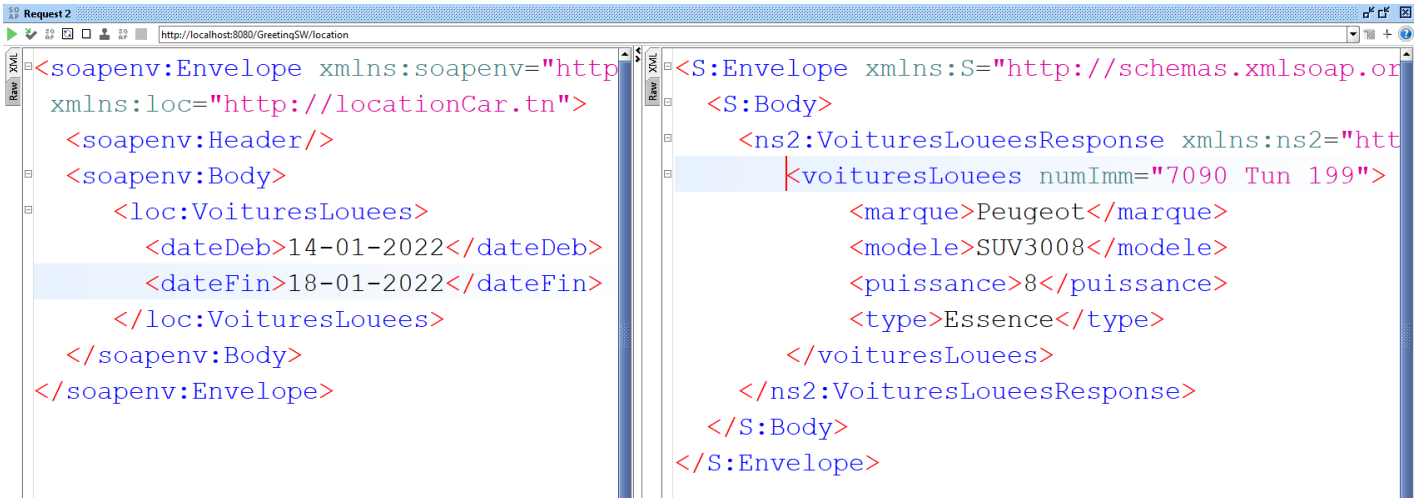


Figure 4: Messages SOAP Recherche Voitures Louées

En se basant sur les messages SOAP- des figures 2, 3 et 4- donnés ci-dessus :

Terminez les parties manquantes du code source : des signatures des méthodes du SW étendu en utilisant l'API JAX-WS, et de l'entité Voiture en utilisant l'API JAX-B.

LocationVoiureInterface.java

```
[1].........(targetNamespace = "[2] ....")

public interface LocationVoiureInterface {

      [3]........
      public String newCar ([4]..... Voiture v);

      [5] .......
      public List<Voiture> VoituresLouees (
                    [6] ....String dateDeb,
                    [7] ... String dateFin);



      [8] .......
      public [9] ... listeLocations ([10] ...);

}
```

Voiture.java

```
[11] ........
public class Voiture {

  [12] ........
  protected String numImm;
  protected String marque;
  protected String modele;
  protected int puissance;
  protected TypeVoiture type;

 enum TypeVoiture{Electrique,
                  Essence,
                  Diesel}
```

2. Exercice2 : GraphQL (7 pts)

Dans un autre contexte, Sami un étudiant à ESPRIT, a commencé à développer l'application décrite ci-dessus. Une fois arrivé à la partie où il doit exposer les fonctionnalités de son application en tant que services web, l'étudiant s'est trouvé à faire un choix entre développer les web services en REST ou bien utiliser GraphQL :

1. Citez 3 différences entre Rest et GraphQl qui aideront peut-être l'étudiant à faire son choix :

   .....

   .....

Après avoir lu vos réponses, l'étudiant a finalement choisi GraphQl pour développer les web services de son application mais il a rencontré quelques difficultés.

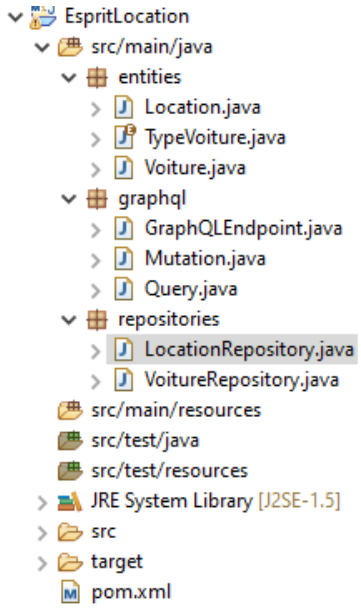


Figure 5: Structure du projet GraphQL

2. En vous référant à la capture de la structure du projet ci-dessus, dites quel fichier manque au projet et citez l'utilité de ce dernier :

   .....

   .....

Votre réponse a été vraiment bénéfique ! Mais Sami a du mal à écrire le contenu du fichier qu'il vient de rajouter au projet.

3. En vous référant à la définition des classes donnée dans la **Figure1** et aux signatures des méthodes suivantes implémentées au niveau du **Repository**:

```
public List<Voiture> getAllCars();
public Voiture getVoitureByMatricule(String immatriculation);
public Voiture createVoiture(Voiture voiture) ;
```

```
public boolean updateVoiture(Voiture voiture)
```

NB : Ci-dessous une capture de test de l'opération **createVoiture** dans postman :

QUERY

```
mutation{
    createVoiture(numImm :"69TU9969", marque:"Volkswagen", modele:"golf 3",
        puissance: 6, type:"Essence"){
        numImm
        modele
    }
}
```

Veuillez compléter le fichier demandé à la place de l'étudiant :

[1] ……Voiture {
numImmatriculation: String
marque:String
modele:String
puissanceFiscale:Int
typeVoiture:[2] ……
}

[3] ……TypeVoiture {
ELECTRIQUE
ESSENCE
DIESEL
}

[4] ……Location {
Id:Int
dateDebut:String
dateFin:String
prix:Float
voiture : Voiture
}

type Query {
[5] ……
[6] ……
}

type Mutation {
[7] ……
[8] ……
}

schema {
[9] …….: [10] ……
[11] ……..: [12] …… }